

Assignment # 01: NCBI Retrieval in Python or Biopython

A from-scratch version of the Entrez/SeqIO exercise, using only the Python standard library

Overview

Biopython's Entrez and SeqIO modules do a lot of invisible work: building the correct NCBI URL, sending the request, and parsing the returned GenBank text into a structured object. In this assignment, you will do all of that by hand using only Python's built-in urllib modules. This forces you to actually understand what a GenBank record looks like as raw text, since you can no longer rely on `record.annotations` or `record.seq` to do the work for you.

Learning Objectives

- Construct a valid NCBI E-utilities URL manually and retrieve data with urllib
- Parse raw GenBank flat-file text using string operations and regular expressions
- Write a correctly formatted FASTA file
- Compute sequence statistics (length, GC%) directly from a plain string
- Handle network and parsing failures gracefully

Assignment Instructions

Write a Python script named `retrieve_sequences_plain.py`:

1. Define a list of at least 5 NCBI nucleotide accession numbers.
2. For each accession, build the E-utilities URL yourself:
`https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=nucleotide&id=<ACCESSION>&rettype=gb&retmode=text`: then fetch it with `urllib.request.urlopen()`.
3. Extract the organism name from the ORGANISM line of the returned text using a regular expression.
4. Extract the nucleotide sequence from the ORIGIN...// block, stripping out line numbers and whitespace, using a regular expression.
5. Write the result to `<accession>.fasta` yourself: a `'>'` header line followed by the sequence wrapped at 70 characters per line.
6. Compute and print the accession, organism, sequence length, and GC content for each record.
7. Wrap each network request in a try/except so one bad accession doesn't stop the rest of the batch.
8. Add a short pause (e.g. `time.sleep(0.4)`) between requests to avoid overloading NCBI's servers.

Submission Requirements

- One Python file: `retrieve_sequences_plain.py`

- The generated .fasta files
- A short paragraph comparing this version to the Biopython version what did Biopython save you from writing?

Grading :

Criterion	Description
Manual HTTP request built correctly	Constructs the correct E-utilities URL and retrieves text via urllib
GenBank text parsed correctly	Organism and sequence correctly extracted using string/regex operations
FASTA files written correctly	Header + sequence wrapped at a consistent line width, one file per accession
Summary statistics accurate	Length and GC% computed correctly by hand
Error handling & code quality	Failed fetches don't crash the script; code is commented and readable